

Practica CI/CD: informe de reflexion

Gabriel Alexander Cordova Solorzano y Jordy Espinoza
26 de julio de 2026 | [Repositorio publico en GitHub](#)

Pipeline npm test - Docker - Trivy - GHCR	Kubernetes RollingUpdate - probes - Secret	Estrategia Blue-Green nativo
---	--	--

1. Por que elegimos Blue-Green

La aplicacion expone **/version** con version, color y hostname. Esto hace posible validar GREEN antes del corte y demostrar de forma determinista que el Service cambia de **slot=blue** a **slot=green**. BLUE permanece levantado, por lo que un smoke test fallido se revierte cambiando nuevamente el selector. Para este laboratorio es mas claro que Canary, que necesita una muestra grande de peticiones para evidenciar una proporcion de trafico.

La estrategia usa solo recursos nativos: dos Deployments y un Service. Su costo es mantener ambas versiones consumiendo recursos. Hay ademas una limitacion especifica de esta app: cada pod tiene su propio JSON. En produccion primero externalizaríamos la base de datos para evitar catalogos divergentes.

Flujo del corte

1. BLUE sirve v1.
2. GREEN v2 arranca y supera readiness.
3. El smoke test valida GREEN.
4. El Service cambia su selector.
5. Rollback = selector de vuelta a BLUE.

2. Que ocurrio con los datos

DB_PATH=/app/data/products.json apunta a un volumen **emptyDir**, cuyo ciclo de vida pertenece al pod. Al crear un producto y eliminar ese pod, el Deployment crea otro con un volumen nuevo y la aplicacion restaura los tres productos semilla. El producto desaparece. Con dos replicas tambien pueden verse catalogos diferentes segun el pod que responda. Es el comportamiento esperado de la practica, no un error que debia corregirse.

3. Metricas DORA propias

Cambio	Commit	Disponible en cluster	Lead time
Runtime distroless 285e565	23-jul 18:57:46	25-jul 21:34:57.195	50 h 37 min
Arranque lento a07b964	26-jul 00:44:14	26-jul 01:09:59.642	25 min 45.643 s

Lead time promedio 25.5246 h	Frecuencia 2 / 2 dias = 1 por dia	Change failure rate 1 / 3 = 33.3 %
---------------------------------	--------------------------------------	---------------------------------------

El CFR incluye un rollout que alcanzo el progress deadline y requirio corregir permisos, mas dos despliegues exitosos con timestamp. Los cambios del selector Blue-Green no se cuentan como nuevas promociones porque no cambian la imagen. Frente a la tabla clasica de clase, una promocion diaria y un lead time promedio cercano a un dia muestran rapidez; el 33.3 % de fallos evidencia que la estabilidad debe mejorar. La muestra es pequena y solo describe este laboratorio.

4. Problemas reales y como los resolvimos

1. Trivy bloqueo la imagen inicial. El run 29980646258 fallo por vulnerabilidades CRITICAL. El runtime final se cambio a distroless; el run 30054762604 termino en verde antes de publicar.

2. El usuario no root no escribia el JSON. Se mantuvo UID/GID 65532 y se agrego fsGroup=65532 para que /app/data sea escribible sin habilitar root ni quitar readOnlyRootFilesystem.

3. La app tarda 12 segundos en estar lista. Durante ese periodo /health responde 503. Readiness tolera el arranque y liveness comienza despues; aumentar replicas no corregiria una sonda mal ajustada.

4. La reproduccion tenia un orden incorrecto. El README aplicaba el Deployment antes de crear el Secret requerido. Se corrigio el orden y se agregaron scripts para el corte y el smoke test.

5. Conclusion

El flujo aplica fail-fast: pruebas, build, escaneo CRITICAL y publicacion SHA/latest. Kubernetes aporta RollingUpdate y un corte Blue-Green reversible. La siguiente mejora prioritaria es externalizar la base de datos e integrar el smoke test como verificacion automatica posterior al despliegue.